



**TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA
SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

SEMESTRE:

ENERO - AGOSTO 2026

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

PATRONES DE DISEÑO

UNIDAD A EVALUAR:

III

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

VELAZQUEZ MORALES LUCERO - 22210362

NOMBRE DE MAESTRO(A):

GUERRERO LUIS MARIBEL

Índice

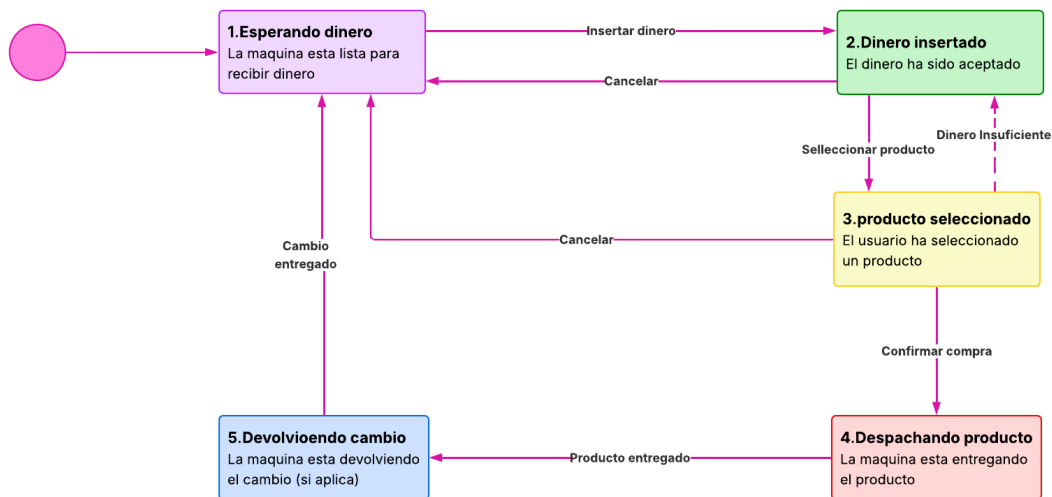
Que es el patrón Estado.....	3
Diagrama de clases.....	3
Diseño.....	4
Código.....	4
Conclusión.....	4

Que es el patrón Estado

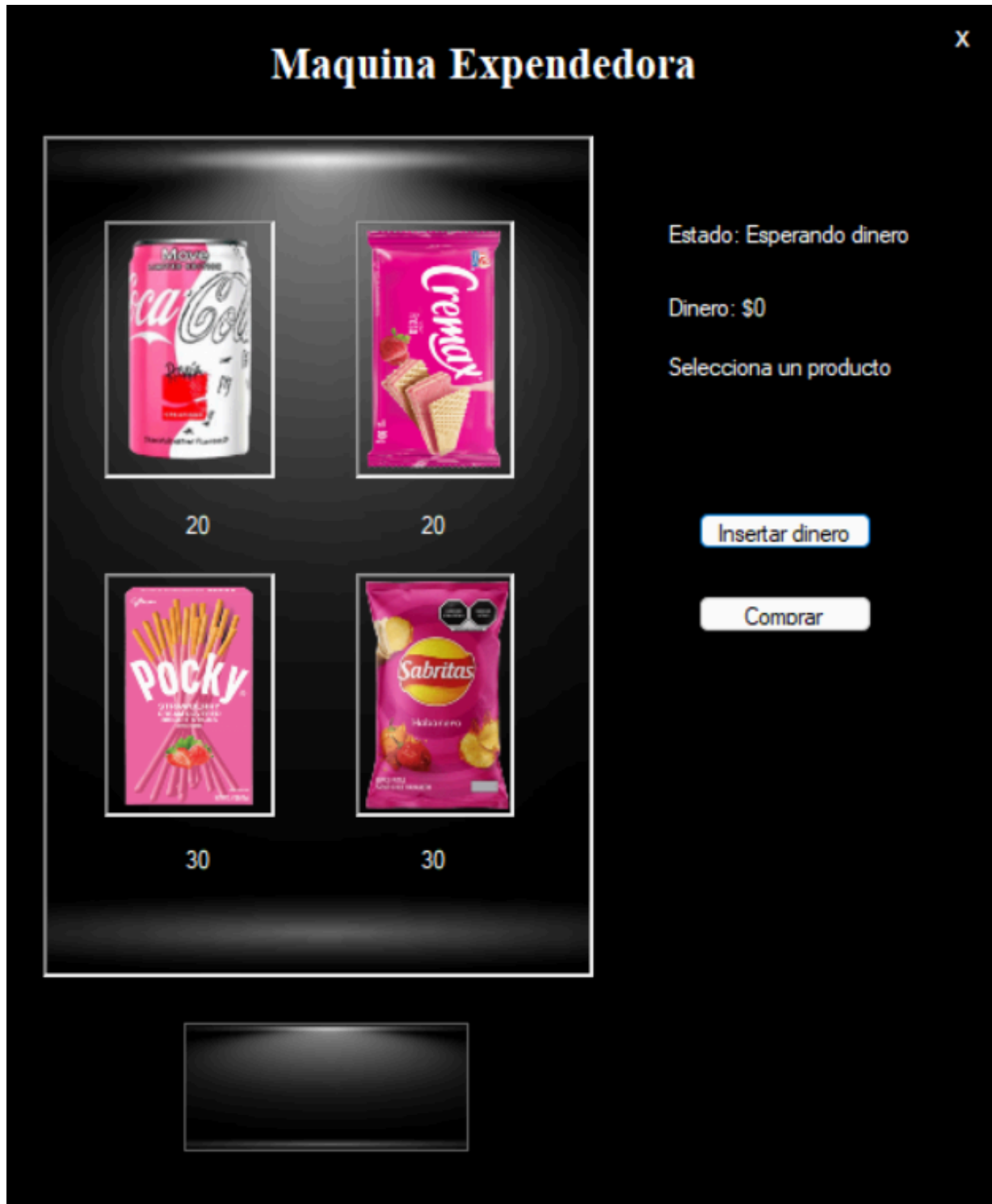
El patrón de estado es un patrón de diseño de software de comportamiento que permite que un objeto altere su comportamiento cuando cambia su estado interno. Este patrón está cerca del concepto de máquinas de estados finitos. El patrón de estado se puede interpretar como un patrón de estrategia, que puede cambiar una estrategia a través de invocaciones de métodos definidos en la interfaz del patrón.

Diagrama de clases

Patron Estado (state) - Maquina expendedora



Diseño



Código

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Media;
using System.Runtime.Remoting.Contexts;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace MaquinaExpendedora
{
    public partial class FrmMaquina : Form
    {
        public FrmMaquina()
        {
            InitializeComponent();
        }

        private void FrmMaquina_Load(object sender, EventArgs e)
        {
            this.StartPosition = FormStartPosition.Manual;
            this.Location = new Point(300, 100);
        }

        string productoSeleccionado = "";
```

```
Contexto contexto = new Contexto();
private void pictureBox1_Click(object sender, EventArgs e)
{

    productoSeleccionado = "Coca";
    lblMensaje.Text = "Seleccionaste Coca-Cola";

    contexto.Producto = productoSeleccionado;
    contexto.Estado.SeleccionarProducto(contexto);

    SoundPlayer player = new SoundPlayer("click.wav");
    player.Play();

}

private void picSabritas_Click(object sender, EventArgs e)
{
    productoSeleccionado = "Sabritas";
    lblMensaje.Text = "Seleccionaste Sabritas";

    contexto.Producto = productoSeleccionado;
    contexto.Estado.SeleccionarProducto(contexto);

    SoundPlayer player = new SoundPlayer("click.wav");
    player.Play();

}

private void picSuavicrema_Click(object sender, EventArgs e)
{
```

```
    productoSeleccionado = "Suavicrema";
    lblMensaje.Text = "Seleccionaste Suavicrema";

    contexto.Producto = productoSeleccionado;
    contexto.Estado.SeleccionarProducto(contexto);

    SoundPlayer player = new SoundPlayer("click.wav");
    player.Play();

}

private void picPocky_Click(object sender, EventArgs e)
{
    productoSeleccionado = "Pocky";
    lblMensaje.Text = "Seleccionaste Pocky";

    contexto.Producto = productoSeleccionado;
    contexto.Estado.SeleccionarProducto(contexto);

    SoundPlayer player = new SoundPlayer("click.wav");
    player.Play();
}

private void lblEstado_Click(object sender, EventArgs e)
{

}

private void btnDinero_Click(object sender, EventArgs e)
```

```
{

    contexto.Estado.InsertarDinero(contexto);

    lblDinero.Text = "Dinero: $" + contexto.Dinero;
    lblEstado.Text = "Estado: Activo";
    lblMensaje.Text = contexto.Mensaje;

    SoundPlayer player = new SoundPlayer("dinero.wav");
    player.Play();
}

private void salir_Click(object sender, EventArgs e)
{
    Close();
}

private void btnComprar_Click(object sender, EventArgs e)
{
    contexto.Estado.ConfirmarCompra(contexto);

    lblMensaje.Text = contexto.Mensaje;
    lblDinero.Text = "Dinero: $" + contexto.Dinero;

}
```



```

        private void panel2_Paint(object sender, PaintEventArgs e)
        {

        }

    }
}

```

//Contexto

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace MaquinaExpendedora
{
    public class Contexto
    {
        public IEstado Estado { get; set; }
        public int Dinero { get; set; }
        public string Producto { get; set; }
        public string Mensaje { get; set; }

        public Contexto()
        {
            Estado = new EstadoEsperandoDinero();
            Dinero = 0;
            Producto = "";
            Mensaje = "Esperando dinero";
        }
    }
}

```

```
}
```

```
//IEstado
```

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
namespace MaquinaExpendedora
```

```
{
```

```
    public interface IEstado
```

```
    {
```

```
        void InsertarDinero(Contexto c);
```

```
        void SeleccionarProducto(Contexto c);
```

```
        void ConfirmarCompra(Contexto c);
```

```
        void Cancelar(Contexto c);
```

```
    }
```

```
}
```

```
//EstadoEsperandoDinero
```

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
using System.Windows.Forms;
```

```
namespace MaquinaExpendedora
```

```
{
```

```
    public class EstadoEsperandoDinero : IEstado
```

```
    {
```

```

    public void InsertarDinero(Contexto c)
    {
        c.Dinero += 10;
        c.Mensaje = "Dinero insertado";
        c.Estado = new EstadoDineroInsertado();
    }

    public void SeleccionarProducto(Contexto c)
    {
        c.Mensaje = "Primero inserta dinero";
    }

    public void ConfirmarCompra(Contexto c)
    {
        c.Mensaje = "No hay dinero";
    }

    public void Cancelar(Contexto c)
    {
        c.Mensaje = "Nada que cancelar";
    }
}

//EstadoDineroInsertado

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

```

```
namespace MaquinaExpendedora
```

```
{
```

```
    public class EstadoDineroInsertado : IEstado
```

```
    {
```

```
        public void InsertarDinero(Contexto c)
```

```
        {
```

```
            c.Dinero += 10;
```

```
            c.Mensaje = "Dinero acumulado: $" + c.Dinero;
```

```
        }
```

```
        public void SeleccionarProducto(Contexto c)
```

```
        {
```

```
            if (c.Producto == "")
```

```
            {
```

```
                c.Mensaje = "Selecciona un producto";
```

```
                return;
```

```
            }
```

```
            c.Mensaje = "Producto seleccionado: " + c.Producto;
```

```
            c.Estado = new EstadoProductoSeleccionado();
```

```
        }
```

```
        public void ConfirmarCompra(Contexto c)
```

```
        {
```

```
            c.Mensaje = "Primero selecciona un producto";
```

```
        }
```

```
        public void Cancelar(Contexto c)
```

```
        {
```

```
            c.Mensaje = "Compra cancelada. Dinero devuelto: $" + c.Dinero;
```

```
        c.Dinero = 0;
        c.Estado = new EstadoEsperandoDinero();
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace MaquinaExpendedora
{
    public class EstadoProductoSeleccionado : IEstado
    {
        public void InsertarDinero(Contexto c)
        {
            c.Dinero += 10;
            c.Mensaje = "Dinero actualizado: $" + c.Dinero;
        }

        public void SeleccionarProducto(Contexto c)
        {
            c.Mensaje = "Producto ya seleccionado";
        }

        public void ConfirmarCompra(Contexto c)
        {
            c.Estado = new EstadoDevolviendoCambio();
        }
    }
}
```

```

        c.Estado.ConfirmarCompra(c);
    }

    public void Cancelar(Contexto c)
    {
        c.Mensaje = "Compra cancelada. Dinero devuelto: $" + c.Dinero;
        c.Dinero = 0;
        c.Estado = new EstadoEsperandoDinero();
    }

    private int ObtenerPrecio(string producto)
    {
        switch (producto)
        {
            case "Coca": return 20;
            case "Sabritas": return 30;
            case "Pocky": return 30;
            case "Suavicrema": return 20;
            default: return 20;
        }
    }
}

```

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace MaquinaExpendedora

```

```

{
    public class EstadoDespachandoProducto : IEstado
    {
        public void InsertarDinero(Contexto c)
        {
            c.Mensaje = "Espere, procesando compra";
        }

        public void SeleccionarProducto(Contexto c)
        {
            c.Mensaje = "Ya se está procesando";
        }

        public void ConfirmarCompra(Contexto c)
        {
            c.Mensaje = "Entregando producto...";
            c.Estado = new EstadoDevolviendoCambio();
        }

        public void Cancelar(Contexto c)
        {
            c.Mensaje = "No se puede cancelar ahora";
        }
    }
}

```

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```
namespace MaquinaExpendedora
{
    public class EstadoDevolviendoCambio : IEstado
    {
        public void InsertarDinero(Contexto c)
        {
            c.Mensaje = "Espere...";
        }

        public void SeleccionarProducto(Contexto c)
        {
            c.Mensaje = "Espere...";
        }

        public void ConfirmarCompra(Contexto c)
        {
            int precio = 0;

            if (c.Producto == "Coca" || c.Producto == "Suavicrema")
                precio = 20;
            else if (c.Producto == "Sabritas" || c.Producto == "Pocky")
                precio = 30;

            int cambio = c.Dinero - precio;

            if (cambio > 0)
                c.Mensaje = "Producto entregado. Cambio: $" + cambio;
            else
                c.Mensaje = "Producto entregado. Sin cambio";
        }
    }
}
```



```
c.Dinero = 0;
c.Producto = "";
c.Estado = new EstadoEsperandoDinero();
}

public void Cancelar(Contexto c)
{
    c.Mensaje = "Operación finalizada";
}
}
}
```

Conclusión

En este proyecto se implementó el patrón de diseño Estado en una máquina expendedora, la implementación del patrón Estado facilitó la comprensión del flujo de la máquina, evitando estructuras complejas de condicionales y mejorando la escalabilidad del programa. Además, permitió simular de manera más realista el funcionamiento de una máquina expendedora, donde cada acción depende del estado actual del sistema.

En conclusión, el uso del patrón Estado demuestra ser una solución eficiente para manejar comportamientos dinámicos en sistemas, mejorando la claridad del código y permitiendo futuras modificaciones o ampliaciones sin afectar la estructura general del proyecto.